

New Session Flow

New Session Flow

Overview

This document defines the **New Session Flow** for your event-driven, Redis-backed, polyglot agent system. It provides the complete end-to-end lifecycle for creating a session, binding it to a frontend tab, initializing the Gateway, and triggering the Agent Harness.

This flow is essential for:

- Starting agent tasks
- Initializing workflow execution
- Opening SSE streams
- Binding UI state to backend session state
- Ensuring future-proof multi-agent orchestration

Folder Structure

```
/frontend
  /api
    createSession.ts
  /state
    sessionStore.ts
  /components
    SessionInitializer.tsx

/gateway
  /routes
    sessions.ts

/agent-harness
  /runtime
    sessionBootstrap.ts
```

File: frontend/api/createSession.ts

```
export async function createSession(agentId: string, input: any) {
  const res = await fetch("http://localhost:3001/sessions/create", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ agentId, input })
  });

  const data = await res.json();
  return data.sessionId;
}
```

File: frontend/state/sessionStore.ts

```
import { EventStreamClient } from "../api/eventStreamClient";

export class SessionStore {
  sessionId: string | null = null;
  eventClient: EventStreamClient | null = null;

  async initSession(agentId: string, input: any) {
    this.sessionId = await createSession(agentId, input);

    this.eventClient = new EventStreamClient();
    this.eventClient.connect(this.sessionId);
  }
}

export const sessionStore = new SessionStore();
```

File: frontend/components/SessionInitializer.tsx

```
import React from "react";
import { sessionStore } from "../state/sessionStore";

export function SessionInitializer({ agentId, input }) {
  const start = async () => {
    await sessionStore.initSession(agentId, input);
  };

  return (
    <button onClick={start}>
      Start Session
    </button>
  );
}
```

File: gateway/routes/sessions.ts

```
import { FastifyInstance } from "fastify";
import { SessionModel } from "../db/SessionModel";
import { TaskModel } from "../db/TaskModel";
import { redisPub } from "../redis/pub";

export default async function sessionsRouter(app: FastifyInstance) {
  app.post("/create", async (req, res) => {
    const { agentId, input } = req.body;

    const sessionId = crypto.randomUUID();

    await SessionModel.create({
      sessionId,
      agentId,
      state: "idle"
    });

    const task = await TaskModel.create({
      agentId,
      input,
      status: "PENDING"
    });

    await redisPub.publish(
      "tasks",
      JSON.stringify({
        type: "task.created",
        taskId: task.id,
        sessionId
      })
    );

    res.send({ sessionId });
  });
}
```

File: agent-harness/runtime/sessionBootstrap.ts

```
import { emitSessionState } from "../events/emitEvents";

export async function bootstrapSession(sessionId: string) {
  await emitSessionState(sessionId, "running");
}
```

Integration Notes

Frontend Responsibilities

- Trigger session creation
- Bind tab to sessionId
- Open SSE stream
- Route events into RxJS bus

Gateway Responsibilities

- Create session record
- Create initial task
- Publish task.created to Redis

Agent Harness Responsibilities

- Receive task.created
- Bootstrap session
- Begin agent execution

Workflow Engine Responsibilities

- Subscribe to session events
- Trigger workflow orchestration

Summary

This New Session Flow provides:

- Deterministic session creation
- Automatic task initialization

- SSE stream binding
- Redis event propagation
- Full integration with Gateway, Agent Harness, and Workflow Engine

It completes the session lifecycle for your MVP and ensures future scalability.